Section A: RDBMS Fundamentals & MySQL Basics (10 Questions)

1. What are the key differences between RDBMS and NoSQL databases?

   RDBMS (Relational Database) stores data in structured tables with fixed schemas, supports SQL queries, and ensures ACID properties. NoSQL databases are schema-less, store data in formats like documents, key-value pairs, or graphs, and focus on scalability and flexibility.

2. Explain the role of **primary keys** and **foreign keys** in RDBMS.

   Primary keys uniquely identify each row in a table, ensuring no duplicates. Foreign keys link rows between tables to maintain referential integrity. For example, a student_id in the enrollments table can be a foreign key referencing students.id.

3. Write a SQL query to create a students table with columns id, name, and age.

```
CREATE TABLE students (
  id INT PRIMARY KEY,
  name VARCHAR(50),
  age INT
);
```

4. What is normalization? Explain with an example.

   Normalization is organizing tables to reduce redundancy. Example: Instead of storing student and course info in one table, separate tables students and courses are created, linked via an enrollments table.

5. What is the difference between CHAR and VARCHAR in MySQL?

   CHAR(n) is fixed-length, padding unused space, while VARCHAR(n) is variable-length and saves space for shorter strings.

6. Write a SQL query to fetch all employees with salary greater than 50,000.

```
SELECT * FROM employees WHERE salary > 50000;
```

7. What are indexes in MySQL, and why are they important?

   Indexes speed up searches on large tables by allowing the database to locate rows faster, similar to a book's index.

8. Debug: A query throws ERROR 1064 (42000). What does this error usually indicate?

   ERROR 1064 usually indicates a syntax error in SQL. Check for missing commas, quotes, or reserved keywords.

9. Explain the difference between **inner join** and **left join**.

   Inner join returns only matching rows from both tables, while left join returns all rows from the left table and matching rows from the right table (NULL if no match).

10. Compare **ACID properties** in RDBMS with an example.

    ACID ensures reliable transactions: Atomicity (all or nothing), Consistency (valid state), Isolation (independent transactions), Durability (permanent). Example: transferring money between accounts.

Section B: DML & DDL Operations in MySQL (10 Questions)

11. Differentiate between DML and DDL with examples.

    DML (Data Manipulation Language) changes data, e.g., INSERT, UPDATE, DELETE. DDL (Data Definition Language) defines or alters schema, e.g., CREATE TABLE, ALTER TABLE.

12. Write SQL queries for:
    a) Insert a new student into the students table.
    b) Update a student's age.
    c) Delete a student with id=5.

    - Insert: INSERT INTO students (id, name, age) VALUES (1, 'Alice', 20);
    - Update: UPDATE students SET age = 21 WHERE id = 1;
    - Delete: DELETE FROM students WHERE id = 5;

13. What is the difference between TRUNCATE and DELETE?

    TRUNCATE removes all rows quickly and resets auto-increment, cannot be rolled back. DELETE removes selected rows, can be rolled back.

14. Write a SQL query to alter the students table and add a new column email.

    ALTER TABLE students ADD COLUMN email VARCHAR(50);

15. How do you create a composite primary key in MySQL?

```
CREATE TABLE enrollment (
  student_id INT,
  course_id INT,
  PRIMARY KEY(student_id, course_id)
);
```

16. Debug: A query says Cannot delete or update a parent row: a foreign key constraint fails. Why?

    Foreign key constraint fails if a child row references a parent row that doesn't exist or you're trying to delete a parent row that is referenced.

17. How do you drop a table in MySQL safely (only if it exists)?

    DROP TABLE IF EXISTS students;

18. What is a **cascade delete** in MySQL? Give an example.

    FOREIGN KEY (student_id) REFERENCES students(id) ON DELETE CASCADE

19. Write a query to count the total number of students grouped by age.

    SELECT age, COUNT(*) FROM students GROUP BY age;

20. Explain the difference between **DDL rollback** and **DML rollback** in transactions.

    DDL rollback is usually not possible (schema changes are permanent), while DML rollback (data changes) can be undone using ROLLBACK in a transaction.

Section C: Designing Data Persistence with SQL (8 Questions)

21. What is data persistence, and why is SQL used for it?

    Data persistence means storing data permanently in a database. SQL provides structured storage, integrity, and query support.

22. How do transactions ensure data persistence in RDBMS?

    Transactions ensure that a set of operations either fully succeed or fail, maintaining data consistency.

23. Compare BEGIN; COMMIT; ROLLBACK; with real-world examples.

    Example: BEGIN; starts transaction, COMMIT; saves changes, ROLLBACK; undoes changes if something fails, like cancelling a money transfer mid-process.

24. Explain how **foreign keys** maintain referential integrity.

    Foreign keys maintain referential integrity by ensuring that child records reference valid parent records.

25. Design a database schema for a **Library Management System** (Books, Members, Loans).

    - Books(id, title, author)
    - Members(id, name, email)
    - Loans(id, book_id FK Books, member_id FK Members, loan_date, return_date)

26. What is a view in SQL, and how does it help in persistence design?

    Views are virtual tables representing query results, simplifying complex queries and ensuring consistency without duplicating data.

27. How do stored procedures improve database persistence logic?

    Stored procedures allow encapsulating business logic in the database, improving consistency and reducing repetitive query code.

28. Explain how indexing impacts data persistence performance.

    Indexing improves read performance but adds a small overhead on writes. Well-indexed tables make persistent storage faster to query.

Section D: Using SQLAlchemy as ORM (6 Questions)

29. What is SQLAlchemy, and why is it considered a powerful ORM in Python?

    SQLAlchemy is a Python ORM that maps database tables to Python classes. It abstracts SQL queries into Python objects, making database interactions easier and safer.

30. Write a SQLAlchemy model for a User table with columns id, username, and email.

```
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
```

```
username = db.Column(db.String(50), unique=True)
email = db.Column(db.String(120))
```

31. How do you query all users with SQLAlchemy ORM?

```
users = User.query.all()
```

32. Explain the difference between db.session.add() and db.session.commit().

db.session.add() adds an object to the session (pending), while db.session.commit() saves all changes to the database.

33. Debug: SQLAlchemy throws No application found. What could be the issue?

No application found usually happens if SQLAlchemy is used outside Flask app context. Make sure app.app_context() is active.

34. Compare raw SQL queries vs SQLAlchemy ORM queries.

Raw SQL is explicit but verbose; SQLAlchemy ORM is concise, safer, and integrates with Python objects.

Section E: Flask Microframework Basics (6 Questions)

35. What is Flask, and why is it called a **microframework**?

Flask is a lightweight web framework because it provides only core web capabilities, letting you choose libraries as needed.

36. Write a simple Flask app with one route /hello returning "Hello World".

```
from flask import Flask
app = Flask(__name__)
@app.route("/hello")
def hello():
    return "Hello World"
if __name__ == "__main__":
    app.run()
```

37. Explain the difference between app.run() and deploying with a WSGI server like Gunicorn.

    app.run() is for development; WSGI servers like Gunicorn or Hypercorn are for production, handling multiple requests efficiently.

38. Debug: A Flask app shows Address already in use. What should you do?

    "Address already in use" happens when the port is busy. Either change the port or stop the other process.

39. How do you enable debug mode in Flask?

    app.run(debug=True)

40. Compare Flask vs Django for building web apps.

    Flask is lightweight and flexible; Django is full-featured with built-in ORM, authentication, and admin. Flask is easier for small apps; Django is better for large projects.

Section F: Flask Project Setup & Routes (5 Questions)

41. How do you initialize a new Flask project structure with app/ and run.py?

    run.py imports app and calls app.run()

    Initialize Flask project:

    - app/ → contains __init__.py, routes, models.
    - run.py → entry point with app.run().

42. Explain the difference between @app.route("/home") with methods=["GET"] and methods=["POST"].

    methods=["GET"] allows only GET requests, methods=["POST"] allows POST. GET fetches data, POST submits data.

43. Write a Flask route /user/<int:id> that returns "User ID: <id>".

    @app.route("/user/<int:id>")

    def user(id):

      return f"User ID: {id}"

44. Debug: Flask route is not recognized. What might be wrong in imports?

    If a route is not recognized, check imports, make sure the module defining the route is imported into the main app.

45. How do you serve static files (CSS/JS) in Flask?

    Serve static files by placing them in app/static/ and referencing in templates: <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">.

Section G: Hypercorn& HTTP/2 in Flask (3 Questions)

46. What is **Hypercorn**, and how does it differ from Gunicorn?

    Hypercorn is an ASGI server supporting HTTP/2 and async apps. Gunicorn is WSGI-only and synchronous by default.

47. How do you run a Flask app with Hypercorn for HTTP/2 support?

    hypercorn app:app --bind 0.0.0.0:5000 --certfile cert.pem --keyfile key.pem

48. Compare HTTP/1.1 vs HTTP/2 in terms of performance for Flask APIs.

    HTTP/2 allows multiplexing multiple requests over a single connection, reducing latency compared to HTTP/1.1 which handles requests sequentially.

Section H: Blueprints & Extensions (2 Questions)

49. What are Flask Blueprints? Write code to register a users blueprint.

    Flask Blueprints help organize routes modularly. Example:

    ```
    from flask import Blueprint
    users_bp = Blueprint("users", __name__)
    @users_bp.route("/list")
    def list_users():
        return "User List"
    app.register_blueprint(users_bp, url_prefix="/users")
    ```

50. Explain the role of Flask Extensions (e.g., Flask-SQLAlchemy, Flask-Login) with an example.

    Flask Extensions add functionality. Example: Flask-SQLAlchemy for ORM, Flask-Login for authentication. They make Flask lightweight but feature-rich when needed.